

Programación (11/06/2026)

SIMULACRO RECUPERACIÓN – TEMAS 1-7

1. (2,5p) Menciona qué elemento/s y lógica es necesaria implementar en la clase *Fruta* para que funcione correctamente el siguiente código y explica por qué:

```
public static void main(String[] args) {  
    Fruta[] frutas = {new Fruta("banana"), new Fruta("manzana"),  
                      new Fruta("pera"), new Fruta("banana")};  
    TreeSet<Fruta> listaFrutas = new TreeSet<>(Arrays.asList(frutas));  
    System.out.println(listaFrutas);  
}
```

- ¿Qué imprimiría el programa suponiendo que la clase *Fruta* solamente tiene un atributo **nombre** y tenemos implementado el método *toString()*?
- Escribe el código necesario para eliminar de la lista a toda fruta que se llame "banana".

2. (2p) Dada la siguiente cabecera de un método:

```
public static String metodo(int edad, String nombre){  
    return "La persona " + nombre + " tiene " + edad + " años.";  
}
```

a) (0,5p) ¿Qué diferencia habría si su visibilidad fuera **private** en lugar de **public**? ¿y si eliminamos **public** a secas?

b) (0,375p) ¿Qué tipo de método es? Explica por qué.

c) (0,375p) ¿Qué devuelve si se le llama de la siguiente manera?

```
.metodo(20, "paco");
```

d) (0,375p) ¿Qué devuelve si recibe los siguientes datos como parámetro?

```
.metodo(25, 20);
```

e) (0,375p) Escribe la cabecera de un método nuevo aplicando **sobrecarga** al método dado.

3. (1,5p) Dada la siguiente función recursiva,

```
public static int funcion_recursiva(int n) {  
    if (n <= 0) {  
        return 0;  
    } else {  
        return -n + funcion_recursiva(--n);  
    }  
}
```

a) (0,25p) Marca sobre el código dado el camino que rompe la recursividad.

b) (1,25p) ¿Qué mostraría el siguiente programa que llama a la función recursiva

4. (1,5p) Dada esta clase *Persona*:

```
public class Persona {  
    public int edad;  
}
```

una clase externa modifica la edad así: ***p.edad = -5;***

- ¿Qué problema representa esto? ¿Cómo aplicarías encapsulación para evitarlo y validar la edad ≥ 0 correctamente?

5. (1p) Explica qué mecanismo tiene *Java* para dar solución a la herencia múltiple de clases, la cual no está permitida.

6. (1,5p) Considera el siguiente código:

```
int suma = 0;  
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 2; j++) {  
        suma += i * j;  
        System.out.println("i: " + i + ", j: " + j + ", suma: " + suma);  
    }  
}
```

¿Qué valores van tomando *i*, *j*, *suma* y las diferentes salidas por pantalla durante la ejecución del programa? Realiza una traza.